

Step 3 — Report Internal Progress Without Breaking the One-Tool Illusion — Solution

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

```
# mcp_server.py (only the new/changed parts shown)

def format_result(result):
    lines = [f"Deep research on: {result.topic}", "", "Revision trace:"]
    for round_info in result.rounds:
        status = "approved" if round_info.approved else "rejected"
        lines.append(f"  Round {round_info.round_number}: {status}")
        for issue in round_info.issues:
            lines.append(f"    - {issue}")
    lines.append("")
    lines.append(f"Approved: {result.approved}")
    lines.append("")
    lines.append("Summary:")
    lines.append(result.final_summary)
    return "\n".join(lines)

@mcp.tool()
def deep_research(topic):
    result = run_pipeline(topic, max_rounds=3)
    return format_result(result)
```

This works and includes the trace. It's missing type hints and a clearer, more scannable status line than a bare `Approved: True / Approved: False`.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — typed, with a real status line

```
# mcp_server.py
from mcp.server.mcpserver import MCPServer

from pipeline import run_pipeline, PipelineResult
from config import load_config
from logging_setup import get_logger

config = load_config()
logger = get_logger(__name__)

mcp = MCPServer("deep-research-service")

def format_result(result: PipelineResult) -> str:
    lines = [f"Deep research on: {result.topic}", "", "Revision trace:"]
    for round_info in result.rounds:
        status = "approved" if round_info.approved else "rejected"
        lines.append(f"  Round {round_info.round_number}: {status}")
        for issue in round_info.issues:
            lines.append(f"    - {issue}")

    lines.append("")
    if result.approved:
        lines.append(f"Status: approved after {result.revision_count} round(s).")
    else:
        lines.append(f"Status: NOT approved -- hit the {result.revision_count}-ro

    lines.append("")
    lines.append("Summary:")
    lines.append(result.final_summary)
    return "\n".join(lines)

@mcp.tool()
def deep_research(topic: str) -> str:
    """Research a topic through a Researcher/Fact-Checker loop. Returns the
    final summary plus a short trace of how many revision rounds it took
```

```

and why each rejected round was rejected."""
logger.info("deep_research called for topic=%r", topic)
result = run_pipeline(topic, max_rounds=config.max_revision_rounds)
logger.info(
    "deep_research finished for topic=%r: approved=%s, rounds=%d",
    topic, result.approved, result.revision_count,
)
return format_result(result)

if __name__ == "__main__":
    logger.info("Starting deep-research-service over stdio")
    mcp.run(transport="stdio")

```

Expected output for the RAG topic (2 rounds, approved on the second):

Deep research on: Retrieval-Augmented Generation (RAG)

Revision trace:

Round 1: rejected

- RAG was introduced by Lewis et al. at Facebook AI Research (now Meta AI) in

Round 2: approved

Status: approved after 2 round(s).

Summary:

Retrieval-Augmented Generation (RAG) was introduced by Lewis et al. at Meta AI (then Facebook AI Research) in 2020. It lets a language model retrieve relevant documents before generating an answer, instead of relying only on what it memorized during training.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — a status marker a caller can spot without reading the whole trace

```
def format_result(result: PipelineResult) -> str:
    lines = [f"Deep research on: {result.topic}"]
    if not result.approved:
        lines.append("[NOT FULLY CONFIRMED -- fact-check cap reached before approval]")
        lines.append("")
        lines.append("Revision trace:")
        for round_info in result.rounds:
            status = "approved" if round_info.approved else "rejected"
            lines.append(f"  Round {round_info.round_number}: {status}")
            for issue in round_info.issues:
                lines.append(f"    - {issue}")

        lines.append("")
    if result.approved:
        lines.append(f"Status: approved after {result.revision_count} round(s).")
    else:
        lines.append(f"Status: NOT approved -- hit the {result.revision_count}-round cap")

    lines.append("")
    lines.append("Summary:")
    lines.append(result.final_summary)
    return "\n".join(lines)
```

Expected output on the "almost nothing checkable" topic, if it hits the cap without approval:

```
Deep research on: my opinion of the color blue
[NOT FULLY CONFIRMED -- fact-check cap reached before approval]

Revision trace:
  Round 1: rejected
    - This is a subjective opinion, not a factual claim that can be verified.
  Round 2: rejected
    - Still framed as opinion; nothing here is independently checkable.
  Round 3: rejected
    - Same issue as prior rounds; no factual claims to confirm.
```

```
Status: NOT approved -- hit the 3-round cap.
```

Summary:

Blue is often associated with calm and openness...

Notice the marker is right at the top, before the trace — a caller who only reads the first two lines already knows this result needs extra caution, instead of having to read the whole trace or check a separate field to find out.

Approach 2 — keeping the full API surface simple: one function, no premature streaming

```
# Deliberately NOT built in this step -- shown here only to name the
# alternative and why it's rejected for now:
#
# @mcp.tool()
# async def deep_research(topic: str, ctx: Context) -> str:
#     await ctx.report_progress(1, 3) # requires a client that listens
#     ...
#
# This uses the mcp SDK's live progress-reporting mechanism. It's a real
# feature, but it only helps a client that specifically asks for progress
# updates and is actively listening for them -- most simple clients (the
# Inspector included, by default) don't, and the notifications don't stick
# around as part of the final result the way this step's trace-in-result
# does. Revisit this only if you later build a client that specifically
# wants live updates during a long call; it isn't needed to satisfy this
# step's actual requirement.
```

This isn't code to run — it's here so the decision this step made (structured result over live progress notifications) is written down next to the alternative, not just asserted in the README.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's trace is complete and correct. Approach 1 adds one thing: a marker at the very top of a not-approved result, so a caller skimming the first line — not the whole trace — still catches that this result needs extra caution. Approach 2 isn't a competing implementation; it documents the mechanism this step chose *not* to use, and exactly why, so the decision is traceable rather than assumed.

Which one should you actually write? Intermediate Approach 1 (with the [NOT FULLY CONFIRMED] marker) — it's a small addition that meaningfully improves how fast a caller can tell a struggling result from a clean one, and it costs nothing. Skip building the live-progress version from

Approach 2 unless a real client of yours specifically needs it; building it without a client that listens for it would be effort spent on a feature nothing can actually observe.